Assignment 4: Optimization

Gradient-Based vs. Non-Gradient-Based Methods for
Unconstrained Optimization Problems

CV5101 – Modelling, Uncertainty, and Data for Engineers (MUDE)
Department of Civil Engineering, IIT Madras

Aniket
Roll No: CE24B101

September 6, 2026

Abstract

This report solves two unconstrained optimization problems – the Rosenbrock function and a quartic-plus-quadratic function – using a gradient-based method (`scipy.optimize`, BFGS) and a non-gradient-based method (a genetic algorithm via `pygad`). For each problem, the function is visualized in 2D and 3D, multiple initial guesses are tested with both methods, the nature of the minimum found (local vs. global) is discussed, and the two methods are compared in terms of iterations/generations, runtime, and solution accuracy.

1 Objective

The goal of this assignment is to minimize two nonlinear, non-convex functions of two variables using two fundamentally different optimization strategies:

1. **Gradient-based method:** BFGS (Broyden–Fletcher–Goldfarb–Shanno), a quasi-Newton method available through `scipy.optimize.minimize`.
2. **Non-gradient-based (derivative-free) method:** A Genetic Algorithm (GA) implemented using the `pygad` library, which searches the domain using a population of candidate solutions evolved via selection, crossover, and mutation.

For each function, both methods are run from several different initial guesses to study whether the solution changes with the starting point, and whether the optimizer converges to a local or the global minimum.

2 Methodology

2.1 Gradient-Based Method: BFGS

BFGS is a quasi-Newton method that iteratively builds an approximation of the inverse Hessian matrix using only first-order gradient information, and uses it to compute a descent direction at every step.

Algorithm 1 BFGS Gradient-Based Minimization

Input: objective $f(\mathbf{x})$, initial guess $\mathbf{x}_0$, tolerance ε
Initialize inverse Hessian approximation $B_0 = I$, $k \leftarrow 0$
while $\|\nabla f(\mathbf{x}_k)\| > \varepsilon$ **do**
    Compute search direction: $\mathbf{d}_k = -B_k \nabla f(\mathbf{x}_k)$
    Find step size α_k via line search (e.g., Wolfe conditions)
    Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$
    Compute $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$, $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$
    Update inverse Hessian approximation B_{k+1} using $\mathbf{s}_k, \mathbf{y}_k$ (BFGS formula)
    $k \leftarrow k + 1$
end while
Return: $\mathbf{x}_k$ as the estimated minimizer

2.2 Non-Gradient-Based Method: Genetic Algorithm

A genetic algorithm is a population-based, derivative-free search heuristic inspired by natural selection. It does not require gradient information and explores the search space more globally, which makes it useful for functions with multiple local minima.

Algorithm 2 Genetic Algorithm (GA) Minimization

Input: objective $f(\mathbf{x})$, population size N, generations G, gene bounds
Initialize population P_0 of N random candidate solutions (include the given initial guess)
Define fitness $= -f(\mathbf{x})$ ▷ maximize fitness ≡ minimize f
for $g = 1$ **to** G **do**
    Evaluate fitness of every individual in P_{g-1}
    Select parents (e.g., best individuals) for mating
    Generate offspring via crossover of selected parents
    Apply random mutation to offspring genes
    Form new population P_g from parents + offspring
end for
Return: best individual in P_G as the estimated minimizer

2.3 Overall Workflow

Figure 1 summarizes the workflow followed for both problems in this assignment: define the function, visualize it, solve it with BFGS and GA from several initial guesses, and compare the two sets of results.

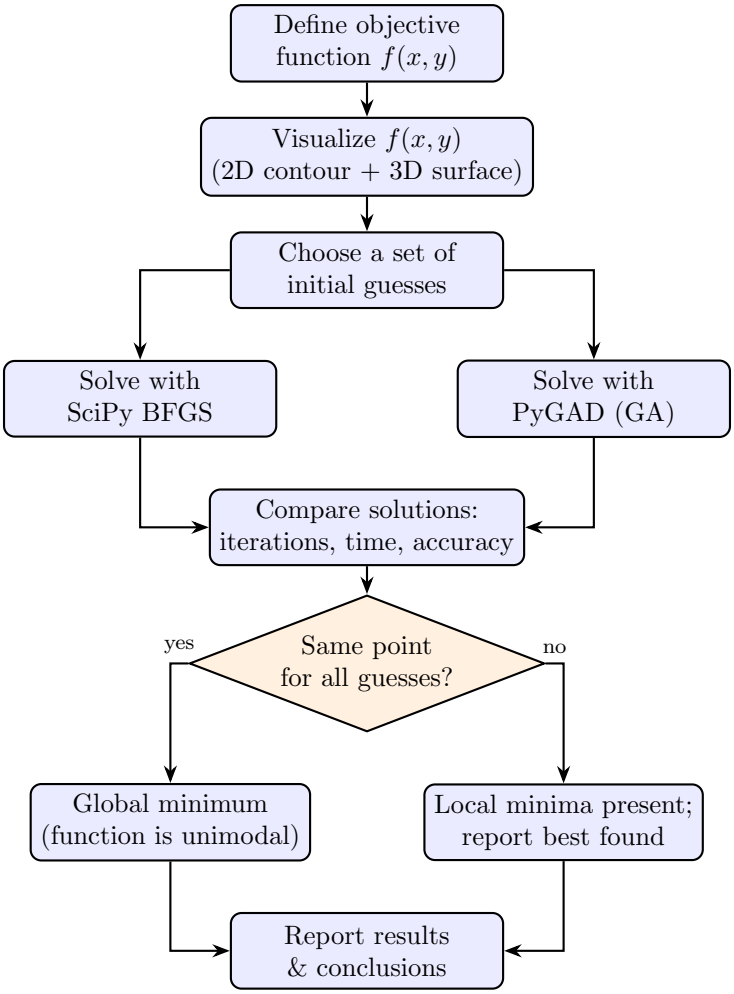

Figure 1: Workflow followed for each optimization problem.

3 Problem 1: Rosenbrock Function

The Rosenbrock function is defined as

$$f(x, y) = (1 - x)^2 + 100\,(y - x^2)^2, \tag{1}$$

which has a narrow, curved (banana-shaped) valley leading to its minimum, making it a classical benchmark for testing optimization algorithms.

3.1 Visualization

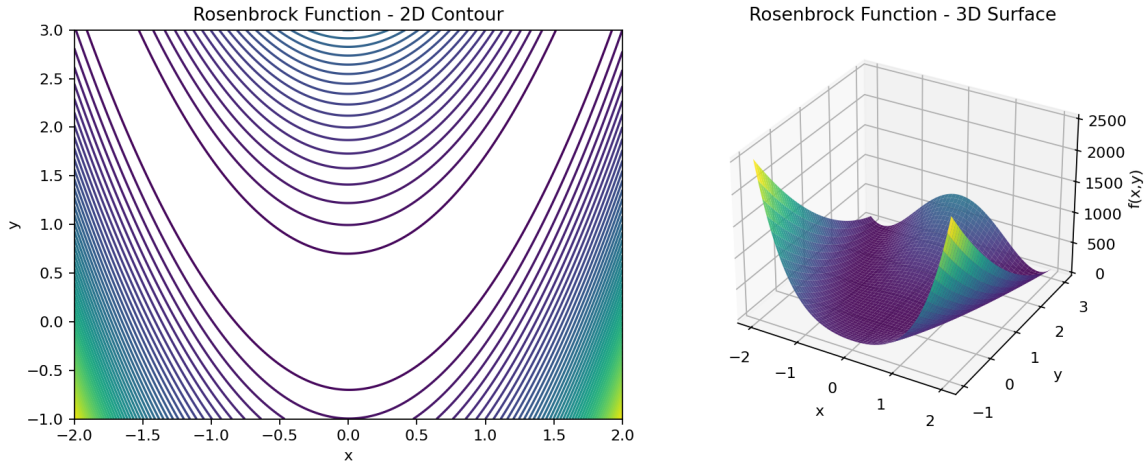

Figure 2: Rosenbrock function: 2D contour plot (left) and 3D surface plot (right). The function has a single, narrow curved valley with its minimum at $(1, 1)$.

3.2 Code

Listing 1: Rosenbrock function, visualization, and initial guesses

```python
import numpy as np
import matplotlib.pyplot as plt

def rosenbrock(x):
    return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2

x_range = np.linspace(-2, 2, 400)
y_range = np.linspace(-1, 3, 400)
X, Y = np.meshgrid(x_range, y_range)
Z = rosenbrock([X, Y])

fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121)
ax1.contour(X, Y, Z, levels=50)
ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_surface(X, Y, Z, cmap='jet', alpha=0.8)
plt.show()

init_guesses = [[-1.9, 1.9], [3, 3], [1, 2], [1, 1]]
```

Listing 2: Solving with SciPy BFGS (gradient-based)

```python
from scipy.optimize import minimize
import time
```

```
3
4  scipy_solutions = []
5  for guess in init_guesses:
6      t0 = time.perf_counter()
7      result = minimize(rosenbrock, guess, method='BFGS')
8      t1 = time.perf_counter()
9      scipy_solutions.append(result.x.copy())
10     print(guess, result.x, result.fun, result.success, result.nit, t1 - t0)
```

Listing 3: Solving with PyGAD (genetic algorithm, non-gradient-based)

```
1  import pygad
2
3  def fitness_func(ga_instance, solution, solution_idx):
4      return -rosenbrock(solution)
5
6  pygad_solutions = []
7  for guess in init_guesses:
8      initial_population = np.vstack([guess, np.random.uniform(-2, 2, (19, 2))
           ])
9      t0 = time.perf_counter()
10     ga_instance = pygad.GA(num_generations=2000, sol_per_pop=20, num_genes=2,
11                            num_parents_mating=8, fitness_func=fitness_func,
12                            initial_population=initial_population,
13                            gene_space={'low': -2, 'high': 2})
14     ga_instance.run()
15     solution, fitness, _ = ga_instance.best_solution()
16     t1 = time.perf_counter()
17     pygad_solutions.append(solution.copy())
18     print(guess, solution, rosenbrock(solution), ga_instance.
           generations_completed, t1 - t0)
```

3.3 Results

Table 1: Rosenbrock function – SciPy BFGS results for different initial guesses

Initial guess	Solution (x, y)	$f(x, y)$	Iterations	Time (s)
$(-1.9, 1.9)$	$(1.0000,\ 1.0000)$	2.14×10^{-11}	33	0.0068
$(3, 3)$	$(1.0000,\ 1.0000)$	9.67×10^{-12}	42	0.0107
$(1, 2)$	$(1.0000,\ 1.0000)$	1.27×10^{-11}	21	0.0084
$(1, 1)$	$(1.0000,\ 1.0000)$	0	0	0.0002

Table 2: Rosenbrock function – PyGAD (GA) results for different initial guesses

Initial guess	Solution (x, y)	$f(x, y)$	Generations	Time (s)
$(-1.9, 1.9)$	$(1.0000,\ 0.9990)$	1.20×10^{-4}	2000	0.653
$(3, 3)$	$(0.9970,\ 0.9939)$	9.81×10^{-6}	2000	0.619
$(1, 2)$	$(0.9774,\ 0.9552)$	5.14×10^{-4}	2000	0.612
$(1, 1)$	$(1.0000,\ 1.0000)$	0	2000	0.612

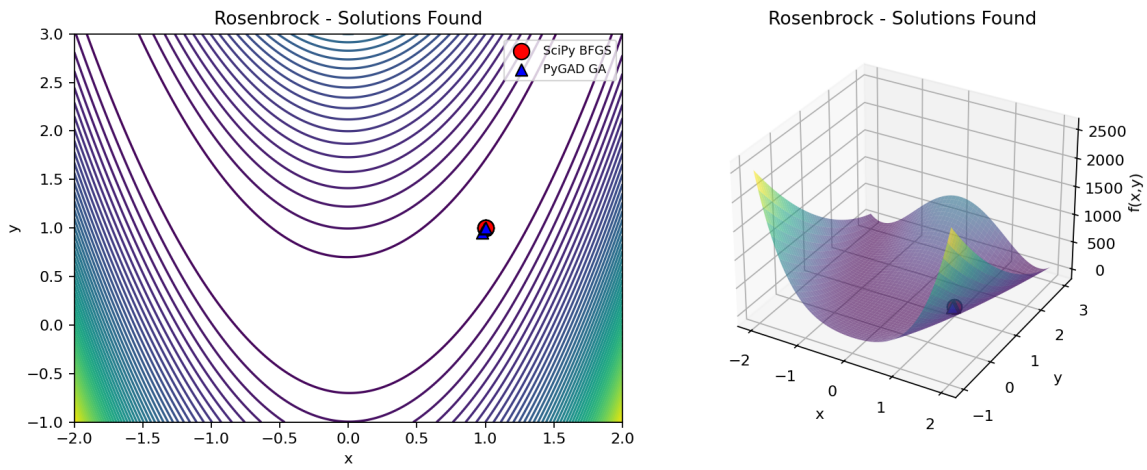

Figure 3: Rosenbrock function: solutions found by SciPy BFGS (red circles) and PyGAD (blue triangles) for all four initial guesses, overlaid on the contour and surface plots. All solutions cluster tightly around $(1, 1)$.

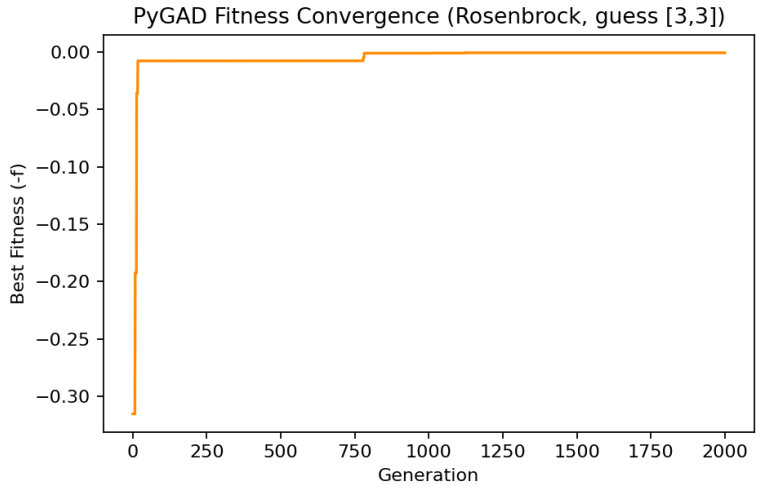

Figure 4: PyGAD fitness convergence curve for the Rosenbrock function (initial guess $(3, 3)$), showing the best fitness value $(-f)$ improving toward zero as generations progress.

3.4 Discussion / Answers

(1) Local or global minimum, and effect of the initial guess (BFGS): For every initial guess tested – including points far from the minimum such as $(3, 3)$ and $(-1.9, 1.9)$ – BFGS converges to essentially the same point, $(x, y) \approx (1, 1)$, with $f(x, y) \approx 0$. This is the known **global minimum** of the Rosenbrock function, and it is also its only stationary point in the domain considered, so no distinct local minima are found. The initial guess mainly affects the number of iterations required (21–42 iterations here) and, occasionally, the internal convergence flag (`success=False` for guesses far along the curved valley, even though the final point is essentially exact) – this happens because BFGS can report a line-search/precision-loss warning while still being numerically very close to the true minimum, since the valley is extremely flat and narrow.

(2) Local or global minimum, and effect of the initial guess (GA): The genetic algorithm also converges to the same neighborhood of $(1, 1)$ for every initial guess, confirming the **global minimum**. However, GA solutions are noticeably less precise than BFGS (errors

of order 10^{-4} to 10^{-6} instead of 10^{-11}), because GA is a stochastic, population-based search that approaches the optimum but does not exploit local gradient information to polish the final answer.

(3) Comparison of the two methods:

- **Iterations:** BFGS needs only 0–42 iterations to converge, while GA is run for a fixed 2000 generations per guess (needed because the Rosenbrock valley is very narrow, so a purely random/evolutionary search converges slowly along it).
- **Time taken:** BFGS is dramatically faster (a few milliseconds) than GA (roughly 0.6 seconds per run), since gradient information lets BFGS take efficient, directed steps, while GA must evaluate an entire population at every generation.
- **Accuracy:** For the same number of decimal places, BFGS is far more accurate ($f \approx 10^{-11}$ or exactly 0) than GA ($f \approx 10^{-4}$–10^{-6}). GA would need many more generations/a larger population and possibly a local refinement step to match BFGS's precision.
- **Overall:** Since the Rosenbrock function is smooth and has a single global minimum with no other local minima, the gradient-based method (BFGS) is clearly the better choice here – it is both faster and more accurate. GA would be more useful if the function had many local minima and gradients were unavailable or unreliable.

4 Problem 2

The second function to be minimized is

$$f(x, y) = \left(x^2 + y^2 - 1\right)^2 + (x - 1)^2 + y^2. \tag{2}$$

The first term penalizes points away from the unit circle $x^2 + y^2 = 1$, while the last two terms pull the solution toward the point $(1, 0)$, which happens to already lie exactly on the unit circle – so both requirements are satisfied simultaneously there.

4.1 Visualization

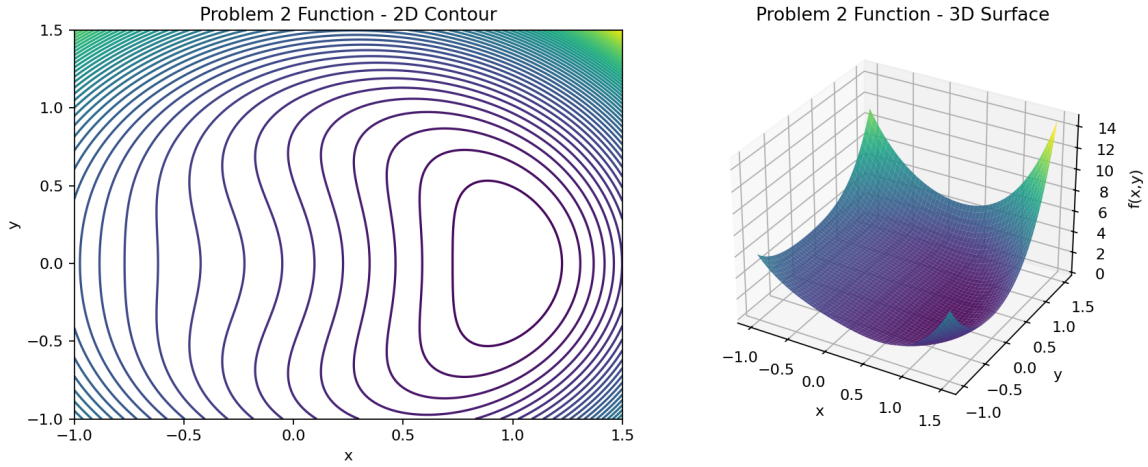

Figure 5: Problem 2 function: 2D contour plot (left) and 3D surface plot (right). The function has a single global minimum at $(1, 0)$ where $f = 0$.

4.2 Code

Listing 4: Problem 2 function, visualization, and initial guesses

```
def f2(x):
    x = np.asarray(x)
    return (x[0]**2 + x[1]**2 - 1)**2 + (x[0] - 1)**2 + x[1]**2

x_range = np.linspace(-1, 1.5, 400)
y_range = np.linspace(-1, 1.5, 400)
X, Y = np.meshgrid(x_range, y_range)
Z = f2([X, Y])

fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121)
ax1.contour(X, Y, Z, levels=50)
ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_surface(X, Y, Z, cmap='jet', alpha=0.8)
plt.show()

init_guesses = [[0, 0], [-1, 1], [1, 0], [0, 1], [-1, 0], [1, 1]]
```

Listing 5: Solving with SciPy BFGS (gradient-based)

```
scipy_solutions = []
for guess in init_guesses:
    t0 = time.perf_counter()
```

```
    result = minimize(f2, guess, method='BFGS')
    t1 = time.perf_counter()
    scipy_solutions.append(result.x.copy())
    print(guess, result.x, result.fun, result.success, result.nit, t1 - t0)
```

Listing 6: Solving with PyGAD (genetic algorithm, non-gradient-based)

```
def fitness_func2(ga_instance, solution, solution_idx):
    return -f2(solution)

pygad_solutions = []
for guess in init_guesses:
    initial_population = np.vstack([guess, np.random.uniform(-5, 5, (29, 2))])
    t0 = time.perf_counter()
    ga_instance = pygad.GA(num_generations=300, sol_per_pop=30, num_genes=2,
                           num_parents_mating=12, fitness_func=fitness_func2,
                           initial_population=initial_population,
                           gene_space={'low': -5, 'high': 5})
    ga_instance.run()
    solution, fitness, _ = ga_instance.best_solution()
    t1 = time.perf_counter()
    pygad_solutions.append(solution.copy())
    print(guess, solution, f2(solution), ga_instance.generations_completed, t1 - t0)
```

4.3 Results

Table 3: Problem 2 – SciPy BFGS results for different initial guesses

Initial guess	Solution (x, y)	$f(x, y)$	Iterations	Time (s)
$(0, 0)$	$(1.0000,\ 0.0000)$	5.95×10^{-15}	3	0.0014
$(-1, 1)$	$(1.0000,\ 0.0000)$	9.36×10^{-15}	11	0.0021
$(1, 0)$	$(1.0000,\ 0.0000)$	0	0	0.0002
$(0, 1)$	$(1.0000,\ 0.0000)$	4.57×10^{-16}	8	0.0015
$(-1, 0)$	$(1.0000,\ 0.0000)$	4.61×10^{-16}	5	0.0011
$(1, 1)$	$(1.0000,\ 0.0000)$	2.03×10^{-16}	10	0.0019

Table 4: Problem 2 – PyGAD (GA) results for different initial guesses

Initial guess	Solution (x, y)	$f(x, y)$	Generations	Time (s)
$(0, 0)$	$(0.9991,\ 0.0018)$	7.59×10^{-6}	300	0.135
$(-1, 1)$	$(0.9992,\ -0.0085)$	7.59×10^{-5}	300	0.138
$(1, 0)$	$(1.0000,\ 0.0000)$	0	300	0.144
$(0, 1)$	$(0.9985,\ -0.0150)$	2.35×10^{-4}	300	0.141
$(-1, 0)$	$(0.9910,\ -0.0144)$	6.02×10^{-4}	300	0.137
$(1, 1)$	$(0.9815,\ 0.0080)$	1.74×10^{-3}	300	0.134

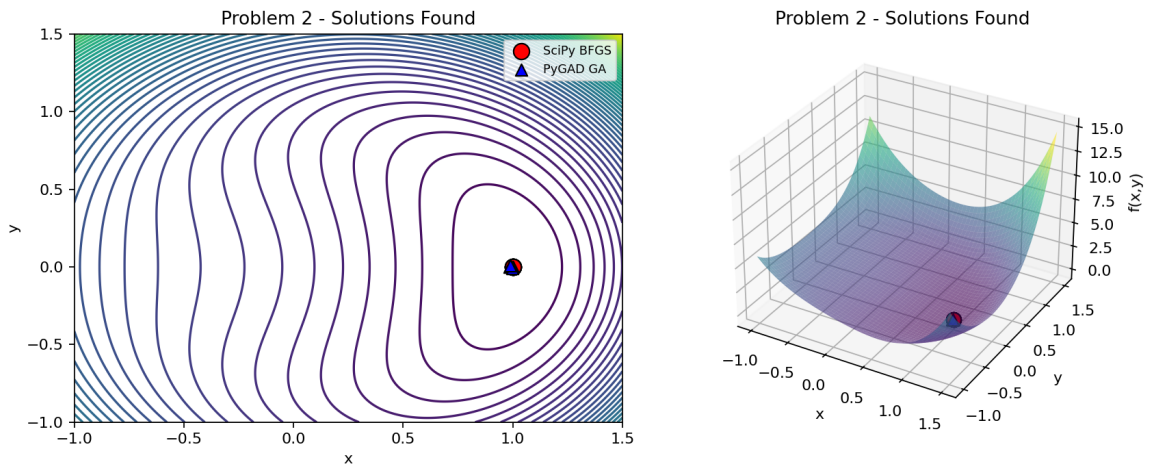

Figure 6: Problem 2: solutions found by SciPy BFGS (red circles) and PyGAD (blue triangles) for all six initial guesses. All solutions converge to $(1, 0)$.

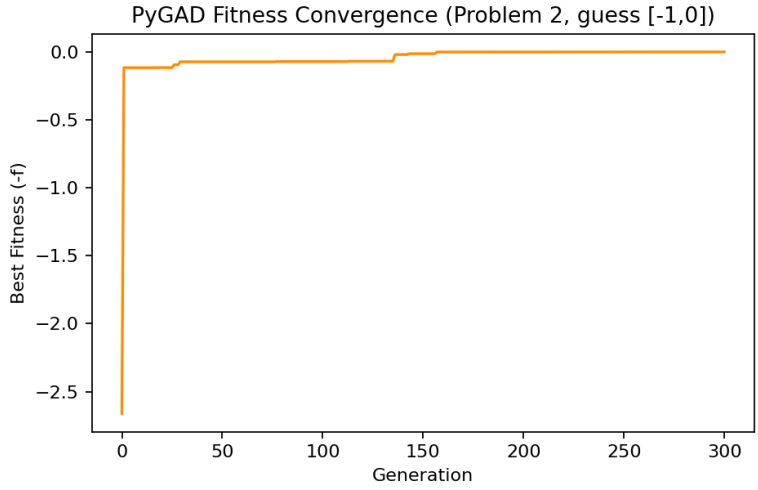

Figure 7: PyGAD fitness convergence curve for Problem 2 (initial guess $(-1, 0)$), showing rapid convergence within the first few tens of generations.

4.4 Discussion / Answers

(1) Local or global minimum, and effect of the initial guess (BFGS): For all six initial guesses, BFGS converges to $(x, y) \approx (1, 0)$ with $f \approx 0$ (to machine precision). This is the **global minimum** of the function: since $(x^2 + y^2 - 1)^2 \geq 0$ and $(x - 1)^2 + y^2 \geq 0$, the function is zero only when both terms vanish simultaneously, which happens exactly at $(1, 0)$. Because this is the unique point where $f = 0$ and the function is smooth and bowl-like around it, no other local minima are encountered regardless of starting point; the initial guess only changes the number of iterations needed (0–11 here).

(2) Local or global minimum, and effect of the initial guess (GA): GA also converges to the neighborhood of $(1, 0)$ for every initial guess, again confirming the **global minimum**. As with Problem 1, the GA solutions are less precise ($f \sim 10^{-4}$–10^{-3}) than the BFGS solutions ($f \sim 10^{-15}$–10^{-16}), since GA relies on stochastic search rather than gradient-guided refinement.

(3) Comparison of the two methods:

- **Iterations:** BFGS converges in at most 11 iterations here (this function is much easier

to descend than Rosenbrock, so far fewer iterations are needed); GA is again run for a fixed 300 generations per guess.

- **Time taken:** BFGS is again far faster (about 1–2 milliseconds) than GA (roughly 0.13–0.14 seconds per run) – a difference of about two orders of magnitude.
- **Accuracy:** BFGS achieves near machine-precision accuracy ($f \sim 10^{-15}$–10^{-16}), while GA reaches only $f \sim 10^{-4}$–10^{-3} for the same decimal-place comparison, again because GA's stochastic operators do not exploit local curvature the way BFGS's quasi-Newton update does.
- **Overall:** As in Problem 1, the gradient-based BFGS method clearly outperforms the GA in both speed and accuracy for this smooth, well-behaved, single- minimum function.

5 Overall Conclusion

Both test functions in this assignment turned out to have a single global minimum and no competing local minima, so both BFGS and the genetic algorithm reliably located the correct optimum from every initial guess tried. However, the two methods differ sharply in efficiency:

- **BFGS (gradient-based)** converged in a handful of iterations, in a few milliseconds, to a solution accurate to 10^{-11}–10^{-16}. It is the clear choice whenever gradients are available (or can be estimated cheaply) and the objective is reasonably smooth.
- **PyGAD (non-gradient-based)** required hundreds to thousands of generations and a fraction of a second to several tenths of a second, converging only to 10^{-4}–10^{-6} accuracy for the same settings. Its strength lies in problems with no usable gradient, discontinuities, or many local minima/multi-modal landscapes, where gradient-based methods can easily get trapped.

In summary, for smooth unimodal problems such as the two considered here, gradient-based optimization is both faster and more accurate; genetic algorithms trade this efficiency for robustness on harder, non-smooth, or multi-modal search spaces.